{EPITECH}

KUBEQUEST_

< BOOTSTRAP />

KUBEQUEST

Docker

You are supposed to be familiar with Docker.

If you're not, follow this tutorial to update and improve your skills.

> It's never too late...

Local Kubernetes sandbox

Minikube is a local K8s cluster, with a single node.
Play around with Minikube until you are ready to rumble.
It can be used as a sandbox to test several commands, such as:

```
minikube start --vm-driver virtualbox
minikube status
kubectl config current-context
kubectl config use-context minikube
minikube delete
```

Microsoft Azure

You can access a laboratory for your group on Microsoft Azure Azure Portal and deploy virtual machines on it.

> Once logged, check your subscription, and search DevTestLab this link, then you can deploy vm.

> We recommend you using terraform to easily re-deploy your virtual machine. Check https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs/resources/dev_test_linux_virtual_mach

Kubernetes basics

With your minikube started, you can now Kubernetes CLI to start using your cluster. We will start by deploying a **pod**, understanding how to interact with it.

Firstly, create a YAML manifest describing your pod, using an hello-world image

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: hello-world
spec:
  containers:
  - name: hello-world
    image: k8s.gcr.io/echoserver:1.4
    ports:
    - containerPort: 8080
```

Save it, create it and try to access your pod properties.

```bash
# Get your pods
kubectl get pods
# Get a pod YAML format
kubectl get pod <pod_name> -o yaml
# Access pod last 50 line of logs
kubectl logs --tail=50 <pod_name>
# Connect a shell
kubectl exec -it <pod_name> -- /bin/sh
```

Finish by deleting your pod. Pod are ephemeral by default, and must be manage by resources like **deployment** or **StatefulSet**.

These resources manage to deploy one or more replicas (pods) of an application. Then you use a **service** to expose your pods internally or externally, selecting pods using labels.

Try a more complex hello-world application using **deployment** and **service**.

With a deployment, you can try capabilities to scale up and scale down, but also to update an image, and perform a rollingUpdate deployment.

```bash
# Get deployments
kubectl get deployment
# Scale deployment to 3 replicas
kubectl scale deployment/<deployment_name> --replicas=3
# Restart a deployment
kubectl rollout restart deployment/<deployment_name>
# Edit a deployment
kubectl edit deployment <deployment_name>
```

In a production mode, your are using an **ingress** to expose your service externally through an **ingress-controller**.

Helm

Kubernetes is a highly-customizable container orchestrator, but for most deployments those yaml are way too verbose. In order to simplify deployment flows, the community built some tools that abstract configurations. Install *Helm* and its bash autocompletion script.

> If not done yet, check out the following tools: *kubens*, *kubectx*.

First deployment

Using *Helm*, deploy a *PostgreSQL* server.

- ✓ Can you list pods running in your cluster ?
- ✓ Can you send an SQL query to your database ?

Take some time to discover the Helm CLI: downgrade, upgrade, release history, undeploy...

Improve

The public Helm chart can be overridden by custom configurations.

- ✓ Can you change the username/password of the database ?
- ✓ For better availability, add 2 slaves to your PostgreSQL server

Now, you should be able to reach PostgreSQL slaves on a dedicated DNS endpoint.

The Helm chart you select might provide advanced PostgreSQL configurations.
Find a way to set the number of max connections To PostgreSQL instance, and the SQL query timeout.

Build

Now, create your first Helm template for the hello-world application you deployed earlier:

- ✓ deployment ;
- ✓ networking ;
- ✓ persistent storage ? ;
- ✓ ...

Make your Helm chart generic: some settings should be customizable for people that are going to use it (instance count, Docker image, exposed port,...)

One more thing

Helm is a very good abstraction, for large infrastructure with dozens of micro-services. In this situation, you will be able to write 1 chart for many similar services.

In the company your working for, what level of abstraction would be appropriated ?

- ✓ 1 chart per language ?
- ✓ 1 chart per application type ?

Bonus: sky is the limit

Helm is nice for creating deployment templates.
But for a single application, you may need to set multiple configurations (for instance, replica count might be different in dev, staging, production).

It's time to take a look on *Kustomize*.

Can you write a generic/default description of your hello-world application ? Then a custom configuration for staging and production ?

v 1.4.1

{EPITECH}